TALOS BRAIN: A VISION-BASED RECURRENT POLICY FOR INSTRUCTION-CONDITIONED NAVIGATION IN AN UNREAL ENGINE SIMULATOR

A CBP PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

MACHINE LEARNING LABORATORY

COURSE CODE: 22PC2CB302

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BUSINESS SYSTEMS

BY

C. CHATRADHARA REDDY	**24071A3214**
SHIVA SAI	**24071A3240**
RAKSHIT CHATURVEDI	**24071A3254**
SAI KOUSHIK REDDY P	**24071A3256**
VEERABHADRA YERRAM	**24071A3262**

UNDER THE GUIDANCE OF

Mr. K. Akhil
Assistant Professor, Department of CSE

Mrs. G. Lakshmi Deepthi
Assistant Professor, Department of CSE

VNRVJIET

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
Vignana Jyothi Nagar, Bachupally, Hyderabad 500118, Telangana, India

November 2026

VNRVJIET
VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
AUTONOMOUS INSTITUTION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the CBP Project titled **"TALOS BRAIN: A VISION-BASED RECURRENT POLICY FOR INSTRUCTION-CONDITIONED NAVIGATION IN AN UNREAL ENGINE SIMULATOR"** is a record of bonafide work submitted by:

C. CHATRADHARA REDDY	**24071A3214**
SHIVA SAI	**24071A3240**
RAKSHIT CHATURVEDI	**24071A3254**
SAI KOUSHIK REDDY P	**24071A3256**
VEERABHADRA YERRAM	**24071A3262**

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B Tech.)** in **COMPUTER SCIENCE AND BUSINESS SYSTEMS** of **Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R25 Academic Regulations**. The work has been carried out under our guidance and supervision at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

The report is original, free from plagiarism, and complies with the Institute guidelines, including those related to academic ethics and AI-generated content.

Signature of Supervisor	**Signature of Supervisor**	**Signature of HOD**
Mr. K. Akhil	Mrs. G. Lakshmi Deepthi	Dr. V Baby
Assistant Professor	Assistant Professor	HOD
Department of CSE	Department of CSE	Department of CSE
VNR VJIET, Hyderabad	VNR VJIET, Hyderabad	VNR VJIET, Hyderabad

VNRVJIET
VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
AUTONOMOUS INSTITUTION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the undersigned, declare that the Project titled **"TALOS BRAIN: A VISION-BASED RECURRENT POLICY FOR INSTRUCTION-CONDITIONED NAVIGATION IN AN UNREAL ENGINE SIMULATOR"** has been carried out by us in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) in COMPUTER SCIENCE AND BUSINESS SYSTEMS of Jawaharlal Nehru Technological University Hyderabad (JNTUH), under VNRVJIET B.Tech. R25 Academic Regulations, at Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad.

We further declare that this work is original, has not been submitted previously for any degree or similar title, and complies with the Institute's plagiarism and academic ethics guidelines, including AI-generated content. The contributions of all team members are clearly indicated in the report.

PLACE: HYDERABAD

DATE: _____ **Signature of the Student(s)**

C. CHATRADHARA REDDY	**24071A3214**
SHIVA SAI	**24071A3240**
RAKSHIT CHATURVEDI	**24071A3254**
SAI KOUSHIK REDDY P	**24071A3256**
VEERABHADRA YERRAM	**24071A3262**

ABSTRACT

Embodied agents must turn raw camera pixels into purposeful action, and a policy that looks perfect in its training room can still fail the moment the room changes. This Community-Based Project presents the design, implementation and honest evaluation of a learned “brain” for the Talos simulator, built on Unreal Engine 5.8.3. The brain is a convolutional-recurrent policy: a convolutional encoder reads a 60 × 80 camera frame, a small network encodes body sense and a short instruction, and a 256-unit gated recurrent unit provides memory. It is trained by behaviour cloning from privileged scripted teachers, while a strict separation ensures that the brain itself never sees privileged state. On 50 held-out seeds the brain reached the crate in 100% of episodes, matching its teacher (100%) and far above a random controller (24%) and a walk-forward rule (30%), with a mean of 25.2 steps against the teacher’s 24.6. Blinding the camera cut success to 14%, confirming that the policy uses vision. The study then tested its own result. In the two-box task the brain scored 100% in its room but only 6% after the box colours were swapped and 28% in an unseen layout, showing that it had memorised positions instead of reading colour. This failure is reported, explained and used to design a colour-only task with randomised, held-out worlds. The main novelty is the evidence-first protocol: fixed held-out seeds, privileged/non-privileged separation, controlled ablations and Wilson confidence intervals, applied in a real engine. The work provides a working baseline for the next roadmap phases and a reusable method for catching shortcut learning early.

Keywords: Embodied AI, Imitation Learning, Behaviour Cloning, Recurrent Policy, Generalisation, Unreal Engine, Instruction-Conditioned Navigation

TABLE OF CONTENTS

TOPIC / CHAPTER TITLE **PAGE NO.**

CERTIFICATE..... 2

DECLARATION.....3

ABSTRACT..... 4

TABLE OF CONTENTS..... 5

CHAPTER 1: INTRODUCTION..... 6

CHAPTER 2: RELATED WORKS..... 8

CHAPTER 3: PROPOSED METHODOLOGY..... 10

CHAPTER 4: RESULTS AND COMPARISON WITH BASELINES..... 14

CHAPTER 5: INFERENCES, LIMITATIONS AND FUTURE SCOPE.....24

REFERENCES.....26

APPENDIX A: CONTRIBUTION OF TEAM MEMBERS..... 29

APPENDIX B: SOFTWARE MODULES AND REPRODUCIBILITY..... 30

APPENDIX C: GLOSSARY OF TERMS.....31

CHAPTER 1 – INTRODUCTION

1.1 Background of the Problem

Embodied artificial intelligence studies agents that perceive and act inside a world through a body, rather than classify pictures that are handed to them. Progress in this field depends on simulators that supply realistic sensing, physics and repeatable tasks [1, 2, 3]. **Talos** is such a simulator, built on Unreal Engine 5.8.3 [4]. It exposes a small, stable interface (the *erl protocol 1.0*): each step of 0.1 s the simulator returns a 160 × 120 camera image together with heading, pitch, velocity and a blocked flag, and accepts four continuous actions (forward, strafe, turn, look) in the range [−1, 1] [5]. A simulator alone, however, does not behave. What the Talos roadmap calls the "brain" is the learned policy that turns what the body sees and feels into what it does next.

This project builds a first version of that brain and, more importantly, measures what it has really learned. A policy that succeeds in the room where it was trained proves little; many apparent successes come from memorising the layout of that room [6, 7]. The work therefore treats evaluation as part of the method and not as an afterthought.

1.2 Problem Statement

Design, train and honestly evaluate a vision-based recurrent policy for the Talos simulator that (i) uses only what a real body could sense, namely the camera image, body sense and, where a task requires it, a short instruction; (ii) reaches a target object in unseen starting conditions faster and more reliably than simple hand-written rules; and (iii) is tested in a way that exposes shortcut behaviour such as memorising object positions, so that the reported success is real.

1.3 Objectives

- Implement the full pipeline in real Unreal Engine: data collection, behaviour cloning, closed-loop evaluation and replay.
- Train a convolutional-recurrent policy ("eyes", "body sense" and a GRU "memory") from teacher demonstrations and compare it with random, walk-forward and privileged-teacher controllers on held-out episodes.
- Extend the policy to an instruction-conditioned task with two boxes and measure what happens when the instruction, the colours or the room layout change.

1.4 Novelty of the Work

- **Evidence-first evaluation.** Every claim is paired with held-out seeds, a stated baseline and a 95% confidence interval. A simulator stub used for debugging is never allowed to count as evidence.
- **Strict privileged/non-privileged split.** The teacher sees true positions; the brain never does. The same separation is enforced when scoring.
- **Failure-driven redesign.** A swap test showed that the first two-box policy fell from 100% to 6% when the colours changed places. This negative result is reported, explained and used to design the colour-only task with held-out worlds.
- **Reproducible real-engine study.** All results come from the real Unreal build, with fixed seed ranges, so every number can be regenerated.

1.5 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 presents the proposed methodology with diagrams, algorithms and equations. Chapter 4 reports the experiments, compares the learned brain with baselines and existing approaches, and discusses the results. Chapter 5 draws inferences and lists limitations and future scope. References and appendices follow.

CHAPTER 2 – RELATED WORKS

2.1 Learning from Demonstrations

Behaviour cloning trains a policy by supervised learning on expert state–action pairs. Pomerleau's ALVINN steered a vehicle from camera input this way [8], and later end-to-end driving work scaled the idea with convolutional networks [9]. Surveys describe its strengths, simplicity and sample efficiency, and its main weakness, compounding error: small mistakes move the agent into states the expert never showed [10, 11]. DAgger addresses this by letting the learner drive and asking the expert to relabel the visited states [12]. We build DAgger into our pipeline but find it unnecessary for the first task, because the teacher data already cover the start poses we test.

2.2 Simulators for Embodied Agents

Habitat, AI2-THOR, DeepMind Lab and CARLA provide scene rendering and agent interfaces for navigation, interaction and driving [1, 2, 13, 3]. They differ in realism and in how much privileged state they expose. Talos shares their goal but is built directly on Unreal Engine [4], keeps the engine reward at zero, and separates privileged diagnostics from the observation. That separation is what lets us train a teacher with true positions and still evaluate a brain that does not have them.

2.3 Visual Navigation and Memory

Deep Q-networks showed that agents can learn control directly from pixels [14]. Target-driven visual navigation learns to reach goals from images [15], and auxiliary-task agents navigate complex mazes from pixels [16]. Because one frame rarely shows the whole situation, recurrent memory is standard: LSTM and GRU cells carry information across steps [17, 18], and recurrent Q-networks handle partial observability [19]. Large-scale reinforcement learning with PPO [20] reaches near-perfect point-goal navigation, but with billions of frames [21]. Our brain follows the same convolution-plus-recurrence design at a much smaller scale and learns from demonstrations instead.

2.4 Instruction-Conditioned Navigation

Vision-and-language navigation conditions behaviour on a natural-language instruction [22], and gated-attention models ground words in visual features [23]. Our instruction is deliberately tiny, a two-element code that picks one of two coloured boxes. This keeps the question clean: does the policy read the instruction and see the colour, or does it fall back on something simpler?

2.5 Generalisation and Evaluation Practice

Reinforcement-learning agents are known to overfit their training levels [6], and imitation learners can latch onto features that correlate with, but do not cause, the expert's action, a failure called causal confusion [7]. Domain randomisation varies the world during training to force robust features [24]. Good practice also asks for multiple seeds, reported uncertainty and shared code [25, 26]. We follow this practice with held-out seeds, held-out layouts and Wilson intervals [27].

2.6 Research Gap

Most reports show the best-case result. Few document, in the same study, a learned policy that scored 100% in its training room, then show how it fell to chance after a controlled swap, and then use that failure to redesign the task. This project supplies that complete loop on a new Unreal-based platform.

CHAPTER 3 – PROPOSED METHODOLOGY

3.1 System Overview

The proposed system couples the Talos simulator with a Python brain runtime through a loopback HTTP link carrying JSON messages. *Figure 3.1* shows the closed loop. At each 0.1 s step the simulator sends the observation; the brain pre-processes the camera frame, runs the policy and returns an action. A second channel carries privileged diagnostics, such as the true agent and target positions. It reaches only the scripted teacher and the scorer, and never the brain.

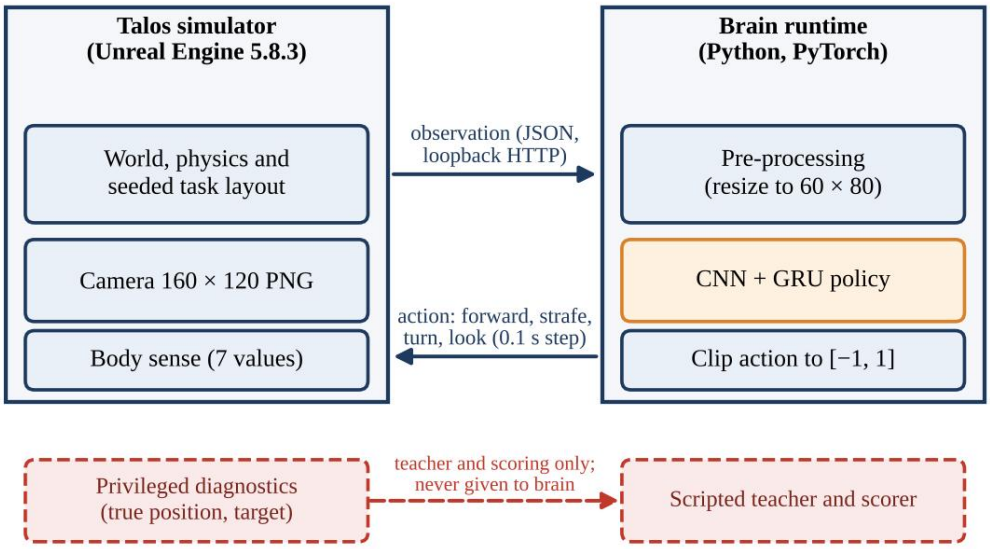


Figure 3.1: Closed-loop architecture of the proposed system and the privileged channel kept away from the brain.

3.2 Problem Formulation

The task is a partially observable decision process. At step t the brain receives $o_t = (I_t, p_t, c)$, where I_t is the 60 × 80 RGB image, $p_t \in R^7$ is the body sense and c is a two-element instruction code (used only in the two-box and colour tasks, which makes the body input nine-dimensional). It outputs a continuous action $a_t \in [-1, 1]^4$. For the crate-reaching task the shaped reward is

$$r_t = \kappa (d_{t-1} - d_t) - 0.005 + 10 \cdot 1[d_t \leq 130 \text{ cm}] \quad (3.1)$$

where d_t is the distance to the crate, κ is a fixed progress scale and the indicator marks success. The engine itself returns zero reward, so this shaping is computed outside it, in the task layer.

The reward is used for scoring and for the optional PPO fine-tuning; the main results use imitation only.

3.3 Network Architecture

Figure 3.2 shows the brain. A convolutional encoder φ turns the image into a feature vector f_t; a small MLP ψ embeds body sense and instruction into e_t. The concatenation feeds a gated recurrent unit with 256 hidden units [18], whose state h_t acts as memory:

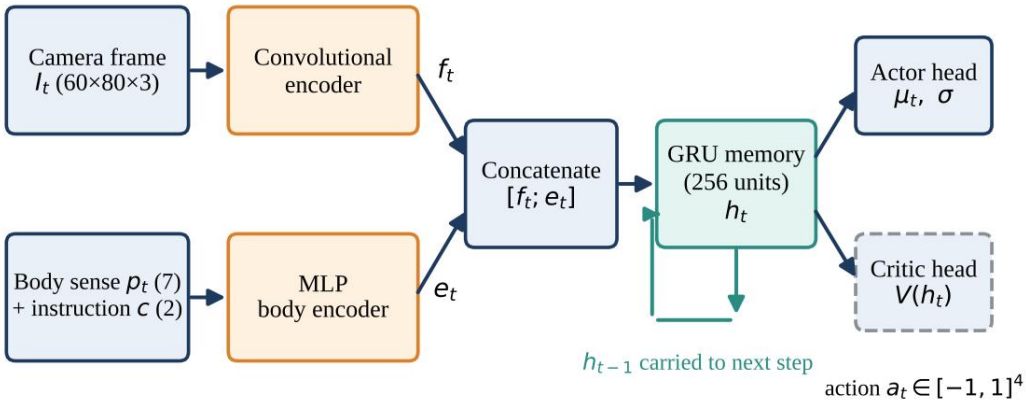


Figure 3.2: Structure of the brain: convolutional eyes, body-sense encoder, GRU memory, actor and critic heads.

$$f_t = \varphi(I_t), \quad e_t = \psi([p_t ; c]), \quad h_t = \mathrm{GRU}(h_{t-1}, [f_t ; e_t]) \tag{3.2}$$

$$\mu_t = W_a h_t + b_a, \quad \pi(a_t \mid h_t) = N(\mu_t, \mathrm{diag}(\sigma^2)), \quad V_t = w_v^T h_t \tag{3.3}$$

The actor head gives the mean action μ_t and a learned spread σ; the critic head V_t is trained only when PPO is used. At run time the deterministic mean, clipped to [−1, 1], is executed. The recurrent state lets the brain remember a box it saw a moment ago while it turns away, which a feed-forward network cannot do [19].

3.4 Teachers and Demonstration Collection

Three scripted teachers produce the demonstrations: a crate teacher, a two-target teacher and a colour teacher. Each reads privileged state and applies a proportional steering law: it turns toward the target in proportion to the heading error and drives forward when the error is small. A scripted start-pose pre-roll diversifies the starting condition before the recording begins. *Algorithm 3.1* lists the collection procedure; the stored episodes contain only what the brain will later see, plus the teacher's action as the label.

Algorithm 3.1: Demonstration collection

1. **Input:** teacher T, seed range S, episode horizon H = 300
2. **for** each seed s in S **do**
   3. reset the simulator with seed s and run the start-pose pre-roll
   4. **for** t = 0 … H−1 **do**
      5. read o_t (image, body sense, instruction) and privileged state z_t
      6. $a^*_t \leftarrow T(z_t)$; store (o_t, a^*_t); step the simulator with a^*_t
      7. **if** the target is reached **then** stop the episode
8. save the episode with its seed and provenance record
9. **return** the episode set

3.5 Behaviour Cloning Objective

The brain is trained on whole episodes so that the recurrent memory sees realistic sequences. For a dataset D of episodes the loss is the mean squared error between the predicted mean action and the teacher's action [8, 9]:

$$L(\theta) = (1 / |D|) \sum_{e \in D} (1 / T_e) \sum_{t=1}^{T_e} \| \mu_t(\theta) - a^*_t \|^2 \qquad (3.4)$$

Parameters are updated with Adam [28] at a learning rate of 3×10^{-4}, batches of 16 episodes and gradient-norm clipping at 1.0, using back-propagation through time in PyTorch [29]. The checkpoint with the lowest validation loss is kept. *Algorithm 3.2* summarises training.

Algorithm 3.2: Behaviour-cloning training

1. **Input:** episode set D, split ratio 90:10 by episode, epochs E
2. split D into D_{train} and D_{val}; initialise θ and Adam
3. **for** epoch = 1 … E **do**
   4. **for** each batch of 16 episodes in D_{train} **do**
      5. unroll the GRU over each episode from $h_0 = 0$; compute L by Eq. (3.4)
      6. clip $\|\nabla L\|$ at 1.0; update θ
   7. compute validation loss on D_{val}; **if** best so far, save checkpoint
8. **return** best checkpoint

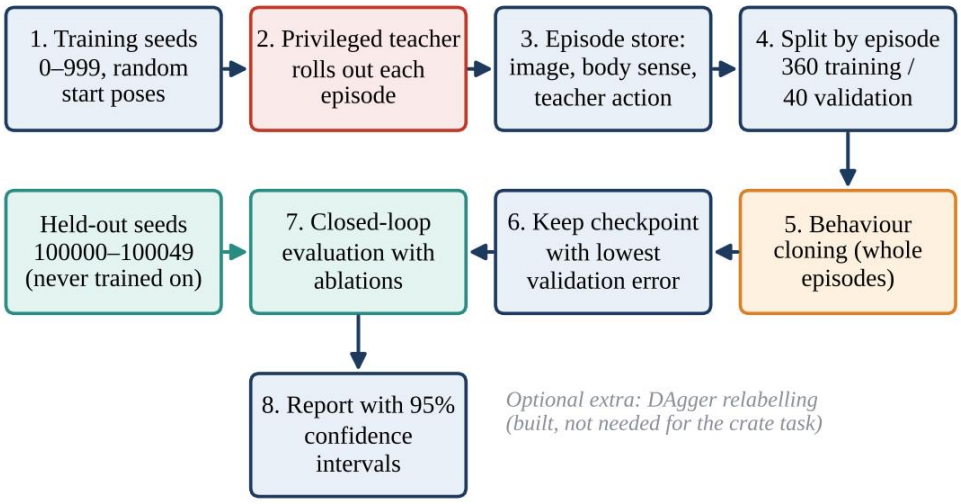


Figure 3.3: End-to-end training and evaluation pipeline with the held-out seed split.

3.6 Evaluation Protocol and Overfitting Controls

Figure 3.3 shows the pipeline. Training uses seeds 0–999 and evaluation uses seeds 100000–100049, which never overlap. A trial counts as a success when the agent ends within 130 cm of the instructed object before the step limit. For a success count k out of n episodes the report gives the rate k/n and the Wilson 95% interval [27]:

$$p = k / n, \quad [\, p + z^2/2n \pm z \sqrt{(\, p(1-p)/n + z^2/4n^2 \,)} \,] / (\, 1 + z^2/n \,), \quad z = 1.96 \qquad (3.5)$$

Each trained policy is compared with a random controller, a walk-forward rule and the privileged teacher (the upper bound). Four controlled ablations probe what the brain relies on: **blind** (camera zeroed), **tight step limits**, **swapped instruction** (the other box is named) and **swapped boxes** (colours exchange places). A fifth test measures performance in a layout absent from training. *Algorithm 3.3* gives the procedure.

Algorithm 3.3: Closed-loop evaluation with ablations

1. **Input:** policy π, controllers {random, walk-forward, teacher}, seeds S_{eval}, condition list K
2. **for** each condition k in K (normal, blind, tight limit, swapped instruction, swapped boxes, unseen layout) **do**
   3. **for** each seed s in S_{eval} **do**
      4. reset with seed s, apply the condition, set $h_0 = 0$
      5. roll out until success or the step limit; record success, wrong-box and steps
   6. report success rate, wrong-box rate, mean steps and the Wilson interval (Eq. 3.5)
7. **return** table of results

CHAPTER 4 – RESULTS AND COMPARISON WITH BASELINES

4.1 Experimental Setup

All results were produced in the real Unreal Engine build of Talos; the lightweight stub simulator used for code debugging is never used as evidence. *Table 4.1* lists the environment. Training and evaluation ran on a Windows laptop with a CUDA-enabled GPU. Each reported figure is a count over 50 held-out episodes unless stated otherwise, so every percentage in this chapter is a multiple of 2%.

Table 4.1: Experimental environment and protocol

Item	Setting
Simulator	Talos on Unreal Engine 5.8.3, erl protocol 1.0
Step interval	0.1 s per step; four actions in [−1, 1]
Observation	160 × 120 RGB frame (resized to 60 × 80), 7-value body sense (+2 instruction values)
Software	Python, PyTorch 2.13.0 (CUDA 12.6)
Training seeds	0–999 (episodes collected from these)
Evaluation seeds	100000–100049 (50 episodes, never used in training)
Success criterion	Ends within 130 cm of the instructed object before the step limit
Default horizon	300 steps; tight-limit tests use 40 and 25 steps

4.2 Data and Training

Table 4.2 summarises the crate-task dataset and training. Four hundred teacher episodes (9,811 frames) were collected in round 0, an average of about 24.5 frames per episode. They were split by episode into 360 for training (8,805 frames) and 40 for validation. The best validation error was 0.00441 mean squared error after 15 epochs, at roughly 1.2 s per epoch on the GPU, so the whole training run takes well under a minute. The small data and short training time are themselves a result: with an informative teacher, a first working brain needs minutes, not days.

Table 4.2: Dataset and training summary for the crate-reaching brain

Quantity	Value
Teacher episodes collected	400 (9,811 frames)
Training / validation episodes	360 / 40 (8,805 training frames)
Optimiser	Adam, learning rate 3×10^{-4}
Batch size, gradient clip	16 episodes, 1.0
Best validation MSE	0.00441 (epoch 15)
Time per epoch	≈ 1.2 s (GPU)

4.3 Experiment 1: Reaching the Crate

The first experiment compares the brain with a random controller, a walk-forward rule (always drive ahead) and the privileged teacher on 50 unseen seeds. *Table 4.3* and *Figure 4.1* show that the brain succeeded in all 50 episodes, matching the teacher, while the random controller and walk-forward rule reached the crate only 24% and 30% of the time. Their intervals do not overlap with the brain's, so the difference is not a matter of chance.

Table 4.3: Crate task on 50 held-out seeds (95% Wilson interval)

Controller	Success (%)	95% interval (%)	Mean steps (successful)
Random	24	[14, 37]	83.3
Walk-forward rule	30	[19, 44]	70.1
Brain (learned)	100	[93, 100]	25.2
Teacher (privileged)	100	[93, 100]	24.6

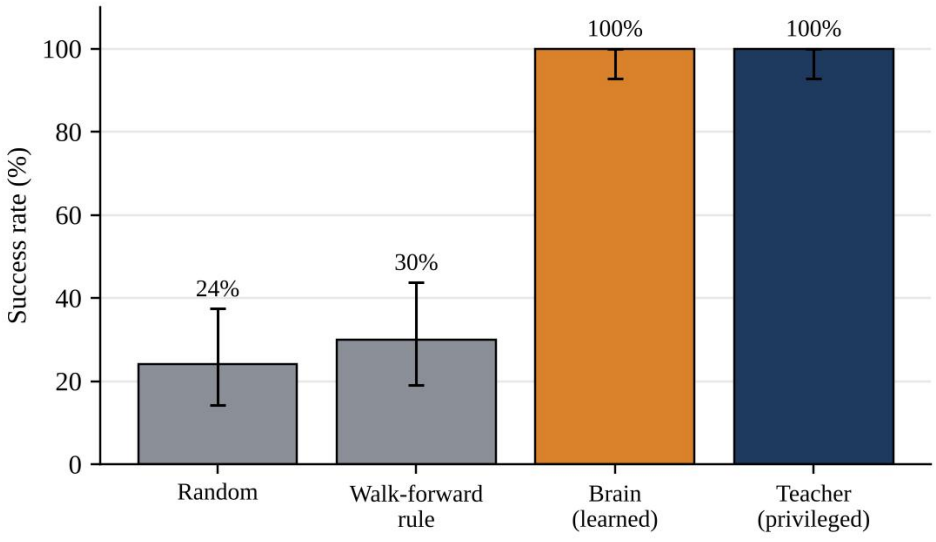


Figure 4.1: Success rate on the crate task with 95% intervals (50 held-out seeds).

Speed tells the same story. *Figure 4.2* shows that the brain needs 25.2 steps on average, within one step of the teacher's 24.6 and about a third of the walk-forward rule's 70.1. The brain has therefore not only found the crate but also learned the teacher's efficient search-and-approach behaviour.

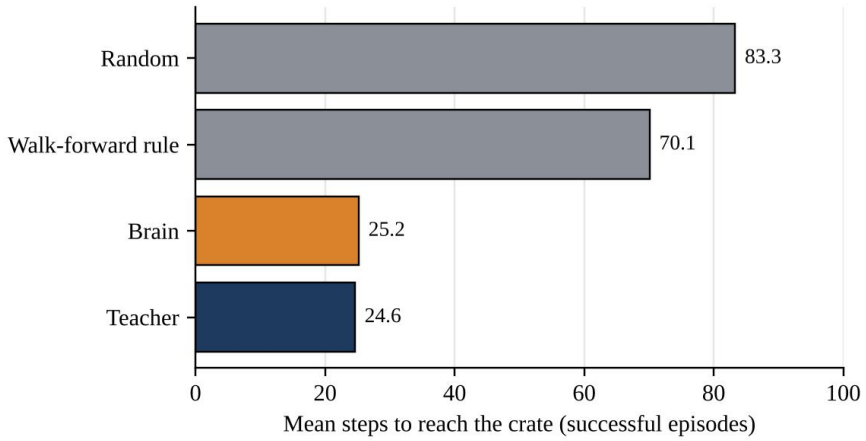


Figure 4.2: Mean steps to reach the crate in successful episodes.

4.4 Experiment 2: Tight Step Budgets

A generous 300-step limit can hide inefficiency. *Figure 4.3* and *Table 4.4* repeat the test with 40 and 25 steps. At 40 steps the teacher succeeds in 96% of episodes and the brain in 92%, against 14% for the walk-forward rule. At 25 steps both fall to 44% because the budget is close to the teacher's own average. The brain tracks its teacher closely at every budget, which shows that the learned policy has imitated efficiency and not just direction.

Table 4.4: Success rate under tight step limits (50 held-out seeds)

Step limit	Teacher (%)	Brain (%)	Walk-forward rule (%)
300 (default)	100	100	30
40 steps	96 [87, 99]	92 [81, 97]	14 [7, 26]
25 steps	44 [31, 58]	44 [31, 58]	–

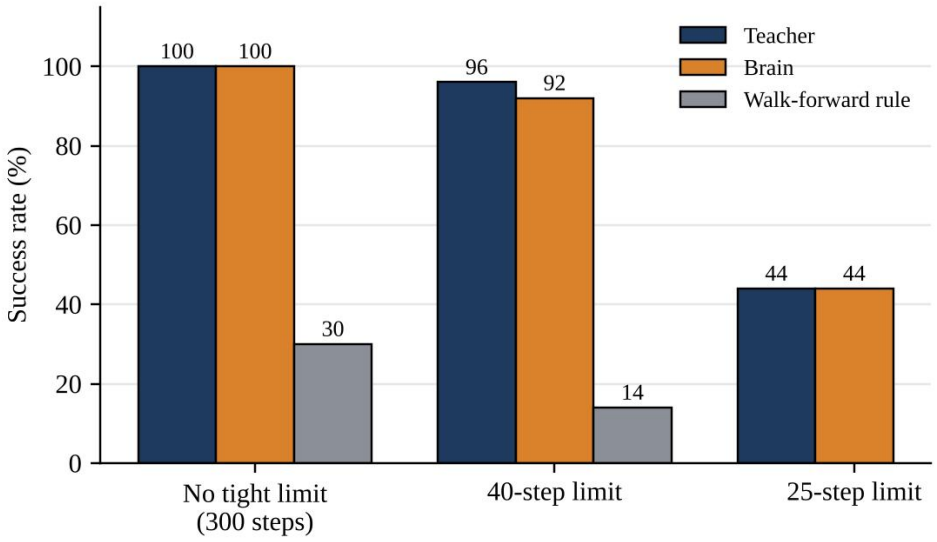


Figure 4.3: Success rate under default, 40-step and 25-step limits.

4.5 Experiment 3: Does the Brain Really Use Vision?

To confirm that the success comes from the camera, the image was replaced by zeros while everything else stayed the same. *Figure 4.4* shows the result: the blind brain succeeded in 14% of 50 episodes, which is no better than the random controller (24%) or the walk-forward rule (30%), although the sighted brain scored 100%. The brain depends on what it sees. This ablation separates a policy that reacts to the crate from one that merely replays a fixed motion.

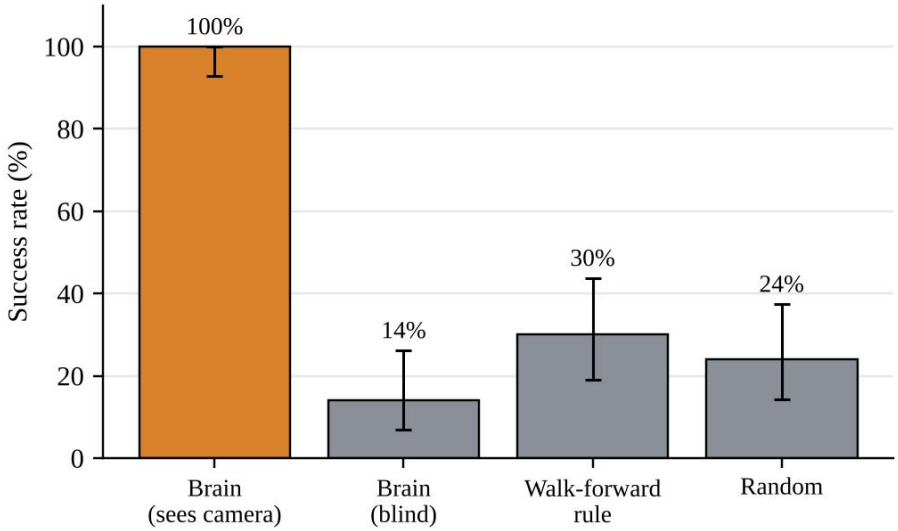


Figure 4.4: Sighted versus blind brain compared with the simple baselines.

4.6 Experiment 4: Instruction-Conditioned Two-Box Task

The second task adds an instruction: the room holds a red and a blue box, and the brain is told which to reach. In the room used for training, the brain and its teacher both succeeded in all 50 episodes, the walk-forward rule in 24% (and in the wrong box in 26%), and the random controller in 2% (*Table 4.5*, *Figure 4.5*). On the surface this is a clean success.

Table 4.5: Two-box task in the training room (50 episodes)

Controller / condition	Correct box (%)	Wrong box (%)	95% interval, correct (%)
Teacher	100	0	[93, 100]
Brain	100	0	[93, 100]
Walk-forward rule	24	26	[14, 37]
Random	2	–	[0, 10]
Brain, swapped instruction	2	98	[0, 10]
Brain, blind	20	–	[11, 33]

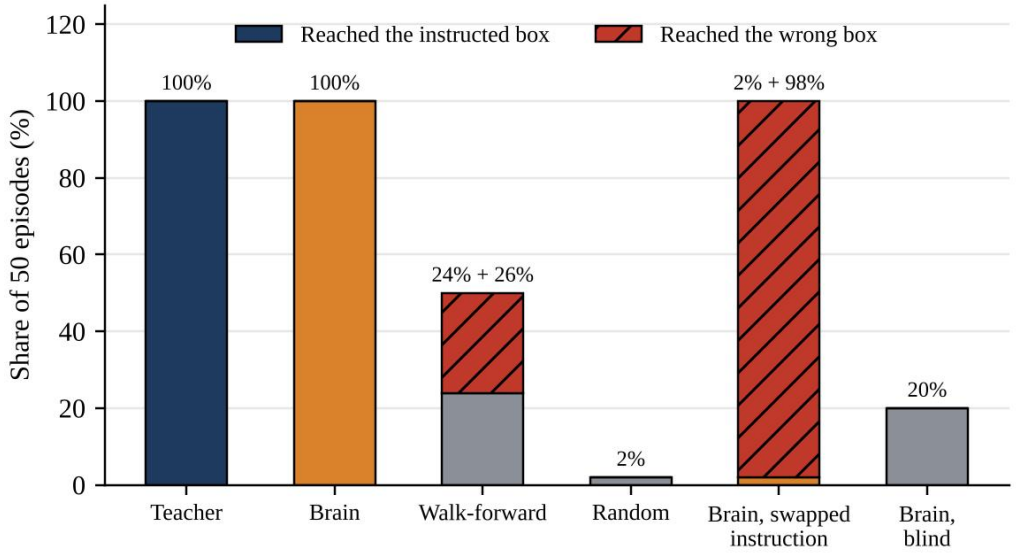


Figure 4.5: Outcomes in the two-box task, including the swapped-instruction test.

The swapped-instruction row is the first warning. When the instruction named the other box, the brain succeeded in only 2% of episodes and went to the wrong box in 98%. A policy that read the instruction would have followed it. This one behaved as if the instruction were irrelevant.

4.7 Experiment 5: Failure Under Swapped Colours

The warning was tested directly by exchanging the colours of the two boxes, leaving their positions unchanged. *Figure 4.6* shows that success dropped from 100% to 6%, with 94% of episodes ending at the wrong box. After the swap the brain walked to the same places as before, so it had been using the room as a map and not the colour as a cue. This is shortcut learning of the kind described by causal-confusion studies [7], and it is exactly the case where an in-room score of 100% is misleading.

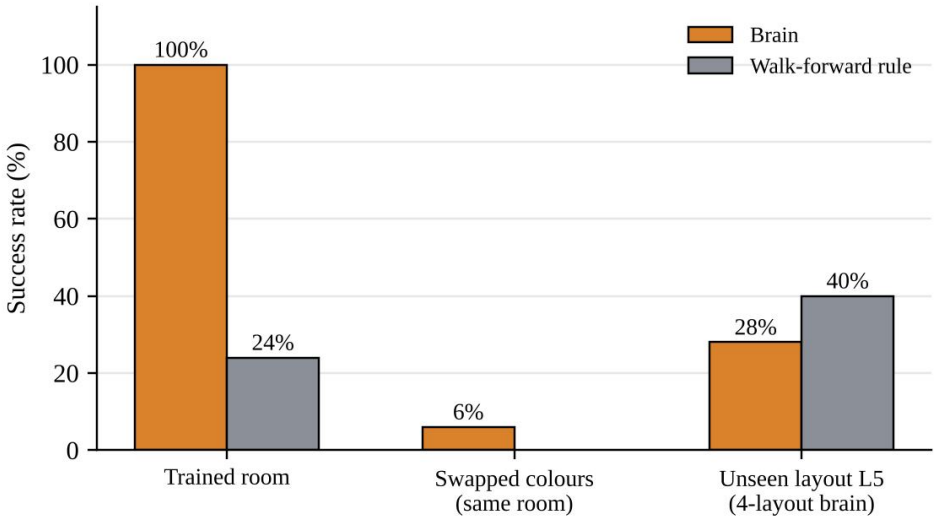


Figure 4.6: Brain success rate falls as the test departs from the training room.

4.8 Experiment 6: Training on Several Layouts, Testing on an Unseen One

A natural remedy is to train on more rooms. A brain was therefore trained on four layouts (L1–L4, validation error 0.00394) and tested on a fifth, L5, that it had never seen. *Table 4.6* and *Figure 4.7* show that it reached the instructed box in only 28% of episodes and went to the wrong box in 72%. The walk-forward rule reached 40% and the privileged teacher 100%. The brain's interval and the walk-forward rule's overlap, so on this test the learned brain is not shown to be better than the simple rule. Swapping the instruction raised its success to 68%, again the wrong direction: the brain was heading to a fixed place, and when the instruction was swapped the named box happened to be that place.

Table 4.6: Unseen layout L5 (50 episodes, brain trained on L1–L4)

Controller / condition	Success (%)	95% interval (%)
Teacher	100	[93, 100]
Walk-forward rule	40	[28, 54]
Brain	28 (72 wrong box)	[17, 42]
Brain, swapped instruction	68	[54, 79]
Brain, blind	44	[31, 58]
Random	0	[0, 7]

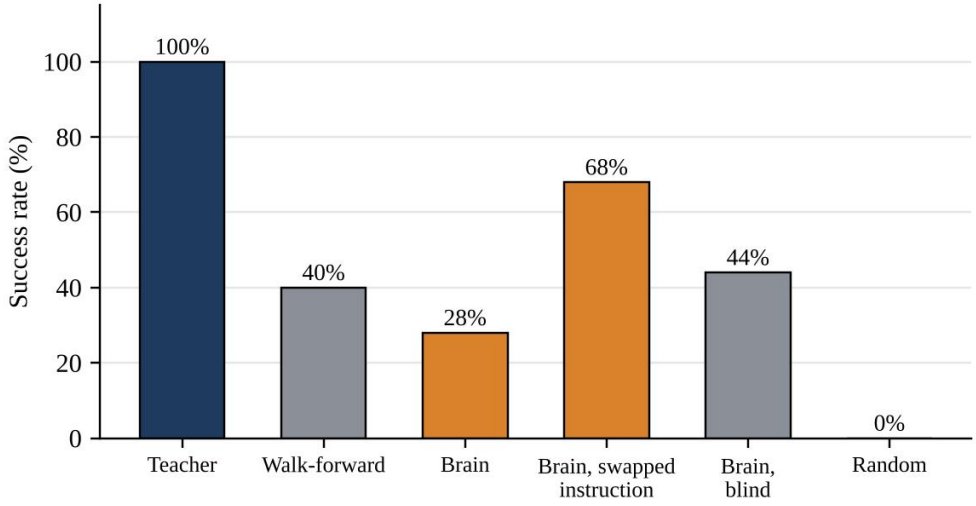


Figure 4.7: Behaviour in the unseen layout L5 for each controller and ablation.

Figure 4.6 places the three conditions side by side. Within its trained room the brain is perfect; change only the colours and it collapses; change the layout and it falls below a trivial rule. Four rooms were still too few, and the brain still found position easier than colour.

4.9 Colour-Only Task: Protocol and Status

The failures above define the next experiment. In the colour-only task (*Figure 4.8*) every world holds two boxes of each colour at positions that change from world to world, and the instruction names only the colour. *Table 4.7* lists the splits. The final-test world is run once, after all design choices are frozen, so that its result cannot be tuned against.

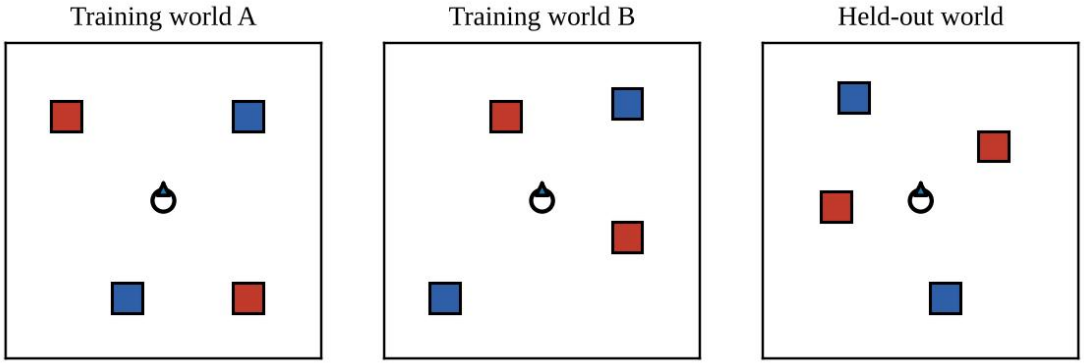


Figure 4.8: Schematic of the colour-only design: positions change between worlds, so only colour identifies the target.

Table 4.7: World splits for the colour-only task

Split	Worlds	Purpose
Training	6	Teacher demonstrations with the colour teacher
Validation	1	Checkpoint selection
Final test	1	Run once; reported as success and wrong-box rate
Spare	2	Reserve for a repeat if the final test is invalidated

Status. The final-test result of this run was not available when this report was written. [Insert the final success rate and the walk-forward rate from the overnight run here.] The design, not a number, is what this report claims for the colour-only task.

4.10 Comparison with Baselines and Existing Approaches

Table 4.8 collects the in-house baselines. The learned brain beats the weak baselines by a wide margin where the test resembles training, equals the privileged teacher, and falls behind a trivial rule when the layout changes. Comparison with published systems cannot be made on numbers, because they use different simulators, tasks and metrics. *Table 4.9* therefore compares design properties.

Table 4.8: Summary of the brain against in-house baselines

Test	Random (%)	Walk-forward (%)	Brain (%)	Teacher (%)
Crate, held-out seeds	24	30	100	100
Crate, 40-step limit	–	14	92	96
Two boxes, trained room	2	24	100	100
Two boxes, swapped colours	–	–	6	–
Two boxes, unseen layout L5	0	40	28	100

Table 4.9: Design comparison with representative existing approaches

Approach	Learning signal	Memory	Held-out test	Typical weakness
ALVINN [8]	Demonstrations	None	Not reported	Compounding error
DAgger [12]	Expert relabelling	Optional	Task dependent	Needs queryable expert
DD-PPO [21]	Reward (RL)	LSTM	Unseen scenes	Billions of frames
Instruction navigation [22]	Demonstrations	Recurrent	Unseen houses	Large data
This work	Teacher demonstrations	GRU	Seeds, swaps, layouts	Position shortcut (found and measured)

Our contribution is not a higher score than these systems but a protocol that detects a failure they would also face. The same swap and unseen-layout checks can be applied to any of them.

4.11 Discussion

Three observations follow from the results. First, vision-based behaviour cloning with a recurrent memory is sufficient for a simple search-and-approach task in Talos, and it reaches teacher-level speed from minutes of training. Second, a perfect in-distribution score can coexist with a total failure to use the intended cue; only controlled swaps expose it. Third, adding a few layouts is not enough to remove the shortcut, which points toward data design (many randomised worlds, colour as the only reliable cue) rather than more training of the same data.

With 50 episodes per test the intervals are wide, for example 14–42% for the 28% result, so small differences should not be over-read; the large gaps (100% against 6%) are robust.

4.12 Summary of Evidence

Table 4.10 restates what each test establishes, so that the reader can see which claims are supported and which are not.

Table 4.10: What each experiment shows

Experiment	Observation	Conclusion supported
Crate, held-out seeds	Brain 100% vs 24–30% for simple rules	The brain learned the task and matches the teacher
Tight step limits	Brain 92% at 40 steps, teacher 96%	Efficiency was imitated, not only direction
Blind ablation	Success falls to 14%	The policy uses the camera
Swapped instruction	2% success, 98% wrong box	The instruction was not used
Swapped colours	6% success, 94% wrong box	Position, not colour, drove behaviour
Unseen layout L5	28% vs 40% for the simple rule	Four layouts did not remove the shortcut
Colour-only task	Design complete; result pending	No claim is made yet

4.13 Threats to Validity

Several factors limit how far the results can be pushed. The test sets are small, with 50 episodes each, which widens the intervals. Each policy was trained from one random initialisation, so run-to-run variation is unknown [25]. The teacher and the brain share the same layouts of boxes in the first experiments, so the strong crate result describes in-distribution behaviour only. The success radius of 130 cm is generous and a stricter radius could lower all scores, although it would do so for every controller alike. Finally, the walk-forward rule and the random controller are weak baselines by design; they show that the task is not trivial, but they are not state-of-the-art navigation methods. We have tried to keep each claim no larger than the evidence behind it, and where the evidence is mixed, as in layout L5, we say so.

CHAPTER 5 – INFERENCES, LIMITATIONS AND FUTURE SCOPE

5.1 Inferences

- A recurrent vision policy learned from privileged-teacher demonstrations can match its teacher in speed and success on the crate task in real Unreal Engine, using 360 training episodes and under a minute of training.
- The policy depends on the camera: blinding it reduces success from 100% to 14%, which is chance level.
- In-room success is a weak indicator of understanding. The two-box brain scored 100% in its room, 6% after a colour swap and 28% in an unseen layout.
- The swap and unseen-layout tests are cheap and decisive. They found the weakness before anyone else could and set the direction for the next design.
- The weakness is one of data and task design, not of the architecture or the simulator, because the teacher solves every variant at 100%.

5.2 Limitations

- **Small evaluation sets.** Each test uses 50 episodes, so intervals are wide and only large effects are conclusive.
- **Single seed of training.** Each policy was trained once; variation across training runs was not measured [25].
- **Imitation only.** The reinforcement-learning phase of the roadmap is specified but not completed; the PPO code is present and untested for results here.
- **Simple tasks.** Two coloured box types in a bounded room are far from open-world navigation, and the instruction is a two-value code, not language.
- **Colour result pending.** The final colour-only test had not been reported when this document was prepared.
- **Debugging stub.** The stub simulator is not evidence, so all numbers depend on access to the Unreal build.

5.3 Future Scope

- Complete the colour-only protocol with many more randomised worlds and domain randomisation of textures and lighting [24], and repeat with several training seeds.
- Use DAgger [12] to correct drift when the learner leaves the teacher's states, and fine-tune with PPO [20] using the reward in Eq. (3.1).
- Add episodic memory [30] for remembering where objects were seen, the next roadmap phase, and predictive learning with a world model [31].
- Study continual learning across tasks and avoid forgetting [32]; finish with a reusable research release of code, seeds and results.

5.4 Recommendations for Future Experiments

- Always include a swap test and an unseen-layout test next to the headline score; they cost little and they decide whether a result can be trusted.
- Report intervals with every rate and avoid ranking controllers whose intervals overlap.
- Freeze the final test world before looking at any result from it, and run it once.
- Keep privileged information out of the brain's inputs and out of its training features, and check this separation in code review.

5.5 Conclusion

This work delivers a working learned brain for Talos and, as importantly, an evaluation method that tells the truth about it. The brain solves the crate task as well as its teacher, depends on vision, and fails exactly where a position shortcut would predict. Reporting that failure and redesigning the task around it is the main outcome. The foundation for the next phases is in place: a pipeline that runs, tests that bite, and a clear next step.

REFERENCES

The following peer-reviewed papers, books, preprints and official documentation form the bibliographic basis of this project.

[1] Savva, M., Kadian, A., Maksymets, O., et al. (2019). "Habitat: A Platform for Embodied AI Research". IEEE/CVF International Conference on Computer Vision (ICCV), pp. 9339–9347.

[2] Kolve, E., Mottaghi, R., Han, W., et al. (2017). "AI2-THOR: An Interactive 3D Environment for Visual AI". arXiv:1712.05474.

[3] Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). "CARLA: An Open Urban Driving Simulator". Proceedings of the 1st Conference on Robot Learning (CoRL), PMLR Vol. 78, pp. 1–16.

[4] Epic Games (2026). "Unreal Engine 5 Documentation". https://dev.epicgames.com/documentation/en-us/unreal-engine/.

[5] Talos project team (2026). "Talos simulator: erl protocol 1.0 specification and phase roadmap (docs/PHASES.md)". Project repository documentation, Talos-main.

[6] Cobbe, K., Klimov, O., Hesse, C., Kim, T., & Schulman, J. (2019). "Quantifying Generalization in Reinforcement Learning". Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR Vol. 97, pp. 1282–1289.

[7] de Haan, P., Jayaraman, D., & Levine, S. (2019). "Causal Confusion in Imitation Learning". Advances in Neural Information Processing Systems (NeurIPS) 32, arXiv:1905.11979.

[8] Pomerleau, D. A. (1989). "ALVINN: An Autonomous Land Vehicle in a Neural Network". Advances in Neural Information Processing Systems 1 (NIPS 1988), Morgan Kaufmann, pp. 305–313.

[9] Bojarski, M., Del Testa, D., Dworakowski, D., et al. (2016). "End to End Learning for Self-Driving Cars". arXiv:1604.07316.

[10] Hussein, A., Gaber, M. M., Elyan, E., & Jayne, C. (2017). "Imitation Learning: A Survey of Learning Methods". ACM Computing Surveys, Vol. 50, No. 2, Article 21.

[11] Osa, T., Pajarinen, J., Neumann, G., Bagnell, J. A., Abbeel, P., & Peters, J. (2018). "An Algorithmic Perspective on Imitation Learning". Foundations and Trends in Robotics, Vol. 7, No. 1–2, pp. 1–179.

[12] Ross, S., Gordon, G., & Bagnell, D. (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR Vol. 15, pp. 627–635.

[13] Beattie, C., Leibo, J. Z., Teplyashin, D., et al. (2016). "DeepMind Lab". arXiv:1612.03801.

[14] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). "Human-level control through deep reinforcement learning". Nature, Vol. 518, pp. 529–533.

[15] Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., & Farhadi, A. (2017). "Target-driven Visual Navigation in Indoor Scenes using Deep Reinforcement Learning". IEEE International Conference on Robotics and Automation (ICRA), pp. 3357–3364.

[16] Mirowski, P., Pascanu, R., Viola, F., et al. (2017). "Learning to Navigate in Complex Environments". International Conference on Learning Representations (ICLR), arXiv:1611.03673.

[17] Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, Vol. 9, No. 8, pp. 1735–1780.

[18] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". Proceedings of EMNLP, pp. 1724–1734.

[19] Hausknecht, M., & Stone, P. (2015). "Deep Recurrent Q-Learning for Partially Observable MDPs". AAAI Fall Symposium Series, arXiv:1507.06527.

[20] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). "Proximal Policy Optimization Algorithms". arXiv:1707.06347.

[21] Wijmans, E., Kadian, A., Morcos, A., et al. (2020). "DD-PPO: Learning Near-Perfect PointGoal Navigators from 2.5 Billion Frames". International Conference on Learning Representations (ICLR), arXiv:1911.00357.

[22] Anderson, P., Wu, Q., Teney, D., et al. (2018). "Vision-and-Language Navigation: Interpreting Visually-Grounded Navigation Instructions in Real Environments". IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3674–3683.

[23] Chaplot, D. S., Sathyendra, K. M., Pasumarthi, R. K., Rajagopal, D., & Salakhutdinov, R. (2018). "Gated-Attention Architectures for Task-Oriented Language Grounding". Proceedings of the AAAI Conference on Artificial Intelligence.

[24] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World". IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 23–30.

[25] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). "Deep Reinforcement Learning That Matters". Proceedings of the AAAI Conference on Artificial Intelligence, arXiv:1709.06560.

[26] Pineau, J., Vincent-Lamarre, P., Sinha, K., et al. (2021). "Improving Reproducibility in Machine Learning Research". Journal of Machine Learning Research, Vol. 22, No. 164, pp. 1–20.

[27] Wilson, E. B. (1927). "Probable Inference, the Law of Succession, and Statistical Inference". Journal of the American Statistical Association, Vol. 22, No. 158, pp. 209–212.

[28] Kingma, D. P., & Ba, J. (2015). "Adam: A Method for Stochastic Optimization". International Conference on Learning Representations (ICLR), arXiv:1412.6980.

[29] Paszke, A., Gross, S., Massa, F., et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". Advances in Neural Information Processing Systems (NeurIPS) 32.

[30] Pritzel, A., Uria, B., Srinivasan, S., et al. (2017). "Neural Episodic Control". Proceedings of the 34th International Conference on Machine Learning (ICML), PMLR Vol. 70, pp. 2827–2836.

[31] Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). "Mastering Diverse Domains through World Models". arXiv:2301.04104.

[32] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. (2017). "Overcoming catastrophic forgetting in neural networks". Proceedings of the National Academy of Sciences, Vol. 114, No. 13, pp. 3521–3526.

APPENDIX A: CONTRIBUTION OF TEAM MEMBERS

This appendix records the division of work among the team members. *Table A.1* lists the members; the responsibilities column is to be completed by the team so that it reflects who actually did each part.

Table A.1: Contribution of team members

Name	Roll number	Responsibilities and contribution
C.Chatradhara Reddy	24071A3214	[To be completed by the team]
Shiva Sai	24071A3240	[To be completed by the team]
Rakshit Chaturvedi	24071A3254	[To be completed by the team]
Sai Koushik Reddy P	24071A3256	[To be completed by the team]
Veerabhadra Yerram	24071A3262	[To be completed by the team]

APPENDIX B: SOFTWARE MODULES AND REPRODUCIBILITY

The brain is implemented as a Python package alongside the Talos repository. *Table B.1* lists the modules. All experiments use fixed seed ranges (training 0–999, evaluation 100000–100049), so each table in Chapter 4 can be regenerated from the stored episodes and checkpoints. The package includes unit tests that run without the engine; engine-based results are produced only on the real Unreal build.

Table B.1: Modules of the brain package

Module	Role
env.py	Wraps the erl protocol: reset, step, observation and action handling
tasks.py	Crate, two-target and colour tasks; rewards; layouts and splits
expert.py	Scripted teachers for each task (use privileged state)
policies.py	Random, walk-forward and learned-policy controllers
collect.py, data.py	Demonstration collection and episode storage with provenance
model.py	Convolutional encoder, body encoder, GRU, actor and critic heads
train_bc.py	Behaviour-cloning training on whole episodes
train_sk.py	Lightweight scikit-learn MLP fallback
train_ppo.py	Optional PPO fine-tuning (not used for reported results)
evaluate.py	Held-out evaluation with ablations and confidence intervals
demo.py, peek.py, look.py	Replay recording and camera inspection
mock_sim.py, smoke.py	Stub simulator and smoke tests for debugging only

The ten colour-task worlds are defined by configuration files (colour_C1 to colour_C10) together with a layout file, and the multi-layout experiments by two_L1 to two_L5 and a layout file.

APPENDIX C: GLOSSARY OF TERMS

Table C.1 defines the terms and abbreviations used in this report.

Table C.1: Terms and abbreviations used in this report

Term	Meaning
Brain	The learned policy that maps camera, body sense and instruction to actions
Teacher	A scripted controller with access to privileged state, used to create demonstrations
Privileged state	True positions and target information that a real body could not sense
Behaviour cloning (BC)	Supervised learning of actions from expert demonstrations
DAgger	Dataset Aggregation: the learner drives and the expert relabels visited states
PPO	Proximal Policy Optimisation, a reinforcement-learning algorithm
GRU	Gated Recurrent Unit, the recurrent memory cell used in the brain
CNN	Convolutional neural network used to encode the camera image
Held-out seeds	Evaluation episodes whose seeds were never used in training
Wilson interval	A confidence interval for a proportion that is reliable at small sample sizes
Swap test	Evaluation with the instruction or the box colours exchanged to detect shortcuts
Wrong-box rate	Share of episodes that end at a box other than the instructed one
MSE	Mean squared error between predicted and teacher actions